1. beadandó feladat: Windows Forms grafikus felületű alkalmazás

Közös követelmények:

- A beadandók dokumentációból, valamint programból állnak, utóbbi csak a megfelelő dokumentáció bemutatásával értékelhető. Csak funkcionálisan teljes, a feladatnak megfelelő, önállóan megvalósított, személyesen bemutatott program fogadható el.

- A megvalósításnak felhasználóbarátnak, és könnyen kezelhetőnek kell lennie. A szerkezetében törekednie kell az objektumorientált szemlélet megtartására.

- A programot háromrétegű (modell/nézet/perzisztencia) architektúrában kell felépíteni, amelyben elkülönül a megjelenítés, az üzleti logika, valamint az adatelérés (amennyiben része a feladatnak). Az egyes rétegeknek megfelelő funkcionalitást kell biztosítania, és megfelelő hierarchiában kell kommunikálnia (pl. a modell csak eseményekkel kommunikálhat a nézettel, a nézet nem végezheti az adatkezelést).

- A modellt eseményvezérelt paradigma szerint kell megvalósítani, azaz a rétegben bekövetkezett változásokról a nézet a modell eseményei révén értesüljön. Amennyiben a feladat időzítést igényel, az időzítő elhelyezését megfelelően kell megválasztani. (Ha a játéklogika szempontjából az idő múlásának pontos ismerete lényeges, akkor az időzítésnek a modellben a helye.)

- A modell működését egységtesztek segítségével kell ellenőrizni. Nem kell teljes körű tesztet végezni, azonban a lényeges funkciókat, és azok hatásait ellenőrizni kell (játékos lépései, nyerési kondíciók, modell eseményeinek kiváltása). Az adatbetöltés/mentés teszteléséhez a perzisztencia működését szimulálni kell.

- A program játékfelületét dinamikusan kell létrehozni futási időben. A megjelenítéshez lehet vezérlőket használni, vagy elemi grafikát. Egyes feladatoknál különböző méretű játéktábla létrehozását kell megvalósítani, ekkor ügyelni kell arra, hogy az ablakméret mindig alkalmazkodjon a játéktábla méretéhez.

- A dokumentációnak jól áttekinthetőnek, megfelelően formázottnak kell lennie, tartalmaznia kell a fejlesztő adatait, a feladatleírást, a feladat elemzését, felhasználói eseteit (UML felhasználói esetek diagrammal), a program szerkezetének leírását (UML osztálydiagrammal), valamint a tesztesetek leírását. A dokumentáció ne tartalmazzon kódrészleteket, illetve képernyőképeket. A megjelenő diagramokat megfelelő szerkesztőeszköz segítségével kell előállítani. A dokumentációt elektronikusan, PDF formátumban kell leadni.

Feladatok:

1. **Pac-Man**

   Készítsünk programot, amellyel a *Pac-Man* játék alábbi változatát játszhatjuk.
   Adott egy n × n elemből álló labirintus, amelynek folyosóin pontok, néhány helyen pedig erőpirulák helyezkednek el. A játékos figurája adott időközönként egy mezőt halad a legutoljára megadott irányba, amíg falba nem ütközik. Ha a játékos olyan irányt ad meg, amerre éppen fal van, a program megjegyzi azt, és a figura az első olyan helyen fordul be, ahol ez lehetséges. A figura a pontokat és az erőpirulákat rálépéssel eszi meg.
   A labirintusban szellemek járnak, amelyek a játékosnál valamivel lassabban mozognak, és kétféle viselkedésűek lehetnek. Az üldöző szellem minden kereszteződésben a játékoshoz vezető legrövidebb út irányába fordul. A kóborló szellem a kereszteződésekben véletlenszerű irányt választ, de nem fordul vissza – hacsak nincs más választása. Ha egy szellem és a játékos egy mezőre kerül vagy helyet cserélnek, a játékos egy életet veszít, és minden figura visszakerül a kezdőpozíciójára. A játékosnak 3 élete van.
   Erőpirula megevése után adott ideig a szellemek megijednek: lelassulnak, a játékostól távolodó irányba mozognak, és a játékos megeheti őket. A megevett szellem visszakerül a kiindulási helyére, és onnan ismét a szokásos módon mozog.
   A pályák méretét és felépítését (a falak, a pontok és az erőpirulák helyzete, a játékos és a szellemek kezdőpozíciója, valamint a szellemek típusa) fájlban tároljuk. A program legalább 3 különböző méretű pályát tartalmazzon.
   A program biztosítson lehetőséget új játék kezdésére a pálya kiválasztásával, valamint játék szüneteltetésére (ekkor nem telik az idő, és nem mozog semmi a játékban). Ismerje fel, ha vége a játéknak: a játékos győz, ha minden pontot megevett, és veszít, ha elfogytak az életei. A program játék közben folyamatosan jelezze ki a játékidőt, a megevett pontok számát, az életek számát, valamint a szellemek ijedtségéből hátralévő időt.

2. **Raktáros**

   Készítsünk programot, amellyel a *Sokoban* játék alábbi változatát játszhatjuk.
   Adott egy n × n elemből álló raktár, amelyet kívülről fal vesz körül, belsejében pedig falak és padló található. A padlón ládák, valamint ugyanennyi célmező helyezkedik el. A raktáros vízszintesen és függőlegesen léphet egyesével. Ha egy ládával szomszédos mezőről a láda felé lép, eltolja azt, amennyiben a láda mögötti mező szabad padló. Egyszerre csak egy ládát tolhat, és ládát húzni nem lehet.
   A játékos célja, hogy minden ládát célmezőre juttasson. A program számolja a lépések és a tolások számát. A játékosnak legyen lehetősége bármikor visszaállni a kezdőállapotra.

Ha egy láda olyan helyzetbe kerül, ahonnan már nem mozdítható, a program jelölje meg a ládát, figyelmeztesse a játékost, és ajánlja fel a kezdőállás visszaállítását. Két ilyen helyzetet kell felismernie: 1) a láda egy sarokban áll, amely nem célmező; 2) a láda egy olyan 2 × 2-es négyzet része, amely csak ládákból és falakból áll, és van benne olyan láda, amely nem célmezőn áll.
A kezdőállást a program új játék kezdésekor véletlenszerűen állítja elő úgy, hogy az biztosan megoldható legyen. Az előállított raktárban minden padlómező elérhető kell legyen a többiből, és egyetlen láda sem állhat induláskor célmezőn. (Megoldható kezdőállás egyszerűen visszafelé játszással állítható elő. A program véletlenszerű belső falakkal legenerálja a raktárat, a ládákat a célmezőkre helyezi, majd a raktárossal adott számú véletlenszerű „húzást" hajt végre. Egy húzásnál a raktáros egy ládával szomszédos mezőről a ládától elfelé lép, és a láda követi őt.)
A program biztosítson lehetőséget új játék kezdésére a raktár méretének megadásával (8 × 8 és 3 láda, 10 × 10 és 4 láda, 12 × 12 és 5 láda), valamint játék szüneteltetésére (ekkor nem telik az idő, és nem léphet a játékos). Ismerje fel, ha vége a játéknak, és jelenítse meg a lépések és a tolások számát, valamint a játékidőt. A program játék közben folyamatosan jelezze ki a játékidőt, valamint a lépések és a tolások számát. Ezen felül szüneteltetés alatt legyen lehetőség a játék elmentésére, valamint betöltésére. A mentésnek az aktuális állás mellett a kezdőállást is tartalmaznia kell, hogy betöltés után is vissza lehessen állni rá.

3. Gyémántbánya

Készítsünk programot, amellyel a *Boulder Dash* játék alábbi változatát játszhatjuk. Adott egy n × n elemből álló bánya, amelynek mezői falak, föld, üres mezők, sziklák, gyémántok, illetve a kijárat lehetnek. A játékos vízszintesen és függőlegesen léphet egyesével. Földre lépve kiássa azt (a mező üres lesz), gyémántra lépve pedig felveszi. Egy sziklát vízszintesen eltolhat, ha a szikla mögötti mező üres.
A bányában gravitáció működik: adott időközönként minden szikla és gyémánt, amely alatt üres mező van, egy mezőt esik. Ha egy szikla vagy gyémánt egy másik szikla vagy gyémánt tetején nyugszik, és mellette, valamint alatta átlósan üres a mező, akkor arra az oldalra legördül (ha mindkét oldal szabad, balra). Ha egy eső szikla vagy gyémánt a játékos mezőjére érkezik, a játékos meghal. A játékos fölött nyugvó szikla viszont nem esik le, amíg a játékos alatta áll.
A bányában ellenfelek is mozognak, amelyek adott időközönként egy mezőt lépnek úgy, hogy a bal kezük felőli falat (vagy más akadályt) követik. Ellenfél csak üres mezőre léphet. Ha egy ellenfél a játékossal (vízszintesen vagy függőlegesen) szomszédos mezőre ér, a játékos meghal. Ha egy szikla egy ellenfélre esik, az felrobban, és a 3 × 3-as környezetében lévő, fal kivételével minden mezőn gyémánt jelenik meg (kivéve a kijáratot).
Ha a játékos összegyűjtötte az előírt számú gyémántot, megnyílik a kijárat, amelyre lépve a játékos teljesíti a pályát. A pályát adott időn belül kell teljesíteni.

A pályák méretét és felépítését (a mezők típusa, a játékos és az ellenfelek kezdőpozíciója, az előírt gyémántszám és az időkorlát) fájlban tároljuk. A program legalább 3 különböző méretű pályát tartalmazzon.
A program biztosítson lehetőséget új játék kezdésére a pálya kiválasztásával, valamint játék szüneteltetésére (ekkor nem telik az idő, és nem mozog semmi a játékban). Ismerje fel, ha vége a játéknak, és jelenítse meg, győzött vagy veszített-e a játékos. A program játék közben folyamatosan jelezze ki a hátralévő időt, valamint az összegyűjtött és az előírt gyémántok számát.

4. Billenő labirintus

Készítsünk programot, amellyel a következő játékot játszhatjuk.
Adott egy n × n elemből álló játékpálya, amely falakból és padlóból áll. A padlón piros és kék golyók, valamint piros és kék lyukak helyezkednek el. A játékos a pályát a négy irány valamelyikébe billentheti. Ekkor az összes golyó egyszerre gurul a megadott irányba, egészen addig, amíg falnak vagy egy másik, már megállt golyónak nem ütközik. Ha egy golyó a saját színével megegyező lyuk fölé ér, beleesik, és eltűnik a pályáról. Ha más színű lyuk fölé ér, a golyó elakad, és a játékos veszít. A golyók gurulását a program animálva jelenítse meg.
A játékos célja, hogy minden golyót a megfelelő lyukba juttasson, minél kevesebb billentéssel. A játékos a lépéseit tetszőleges számban visszavonhatja. A visszavonás is lépésnek számít.
A pályák méretét és felépítését (a falak, golyók és lyukak helyzete) fájlban tároljuk. A program legalább 3 különböző méretű pályát tartalmazzon. A program pályánként őrizze meg a legjobb eredményt, azaz a legkevesebb billentés számát, amely a program újraindítása után is legyen elérhető.
A program biztosítson lehetőséget új játék kezdésére a pálya kiválasztásával, valamint játék szüneteltetésére (ekkor nem telik az idő, és nem lehet billenteni). Ismerje fel, ha vége a játéknak, és jelenítse meg, győzött vagy veszített-e a játékos. Győzelem esetén jelezze, ha a játékos új rekordot ért el. A program játék közben folyamatosan jelezze ki a játékidőt és a billentések számát.

5. Festőgolyó

Készítsünk programot, amellyel a *Roller Splat!* mobiljáték alábbi, kibővített változatát játszhatjuk. Adott egy n × n elemből álló játékpálya, amely falakból és padlóból áll. A padló egy mezőjén egy festőgolyó áll, amelynek kezdőmezője már ki van festve. A játékos megadja a négy irány valamelyikét, a golyó pedig arra gurul, amíg falba vagy zárt kapuba nem ütközik, és útközben minden érintett mezőt kifest. Ha a golyó a megadott irányba nem tud elmozdulni, a lépés nem számít. A golyó gurulását a program animálva jelenítse meg.
A pályán különleges mezők, kapcsolók és kapuk is előfordulhatnak. Ha a golyó áthalad egy kapcsolón, vagy megáll rajta, a pályán lévő összes kapu állapota

megfordul. A zárt kapu falként viselkedik, a nyitott kapun a golyó áthaladhat. A kapumezőket nem kell kifesteni.
A játékos célja, hogy a pálya összes padlómezőjét kifesse, minél kevesebb lépéssel. A játékos a pályát bármikor újrakezdheti.
A pályák méretét és felépítését (a falak, a különleges mezők és a golyó kezdőpozíciója) fájlban tároljuk. A program legalább 3 különböző méretű pályát tartalmazzon. A program pályánként őrizze meg a legjobb elért eredményt, amely a program újraindítása után is legyen elérhető.
A program biztosítson lehetőséget új játék kezdésére a pálya kiválasztásával, valamint játék szüneteltetésére (ekkor nem telik az idő, és nem lehet lépni). Ismerje fel, ha vége a játéknak, és jelenítse meg hány lépéssel és mennyi idő alatt nyert a játékos. A program játék közben folyamatosan jelezze ki a játékidőt, a felhasznált lépések számát, valamint a kifestett terület arányát.

6. Képrejtvény

Készítsünk programot, amellyel a nonogram (más néven *Hanjie* vagy *Griddlers*) japán képrejtvényt oldhatjuk meg. Adott egy n × n mezőből álló tábla, amely egy rejtett fekete-fehér képet tartalmaz. Minden sor bal oldalán és minden oszlop fölött egy számsor áll, amely sorrendben megadja, hogy az adott sorban vagy oszlopban milyen hosszú, összefüggő fekete szakaszok találhatók. A szakaszok között legalább egy fehér mező van. A játékos célja, hogy a számsorok alapján megtalálja a kép összes fekete mezőjét.
A játékos egy mezőt kitölthet (feketének jelöl), vagy kizárhat (fehérnek jelöl, X-szel). Egérrel a bal gomb kitölt, a jobb gomb kizár. Húzással egy sorban vagy oszlopban több mező is jelölhető egyszerre.
A kitöltést a program azonnal összeveti a képpel. Ha a mező a képen fehér, a kitöltés hibás: a mező nem lesz fekete, hanem automatikusan kizárt lesz, és a játékos egy életet veszít. A játékosnak 3 élete van. A kizárás csak a játékos jelölése, ezt a program nem ellenőrzi. Ha egy sor vagy oszlop összes fekete mezőjét megtalálta a játékos, a program halványítsa el az ahhoz tartozó számsort, és a sor vagy oszlop maradék mezőit jelölje kizártnak.
A képeket fájlban tároljuk, a program méretenként legalább 3 képet tartalmazzon. Ezen felül legyen lehetőség adott kitöltési arányú véletlenszerű kép előállítására is.
A program biztosítson lehetőséget új játék kezdésére a táblaméret (5 × 5, 10 × 10, 15 × 15) és a kép megadásával, valamint játék szüneteltetésére (ekkor nem telik az idő, és a tábla nem látható). Ismerje fel, ha vége a játéknak: a játékos győz, ha minden fekete mezőt megtalált, és veszít, ha elfogytak az életei. A program játék közben folyamatosan jelezze ki a játékidőt és az életek számát. Ezen felül legyen lehetőség a játék elmentésére, valamint betöltésére.

7. Baktériumtenyészet

Készítsünk programot, amellyel az *Ataxx* játék alábbi változatát játszhatjuk.
Adott egy n × n mezőből álló tábla, amely egy táptalajt jelképez. Kezdetben mindkét játékosnak két-két baktériuma van, amelyek a tábla átellenes sarkaiban helyezkednek el. A táblán ezen felül néhány terméketlen mező is található, amelyekre nem lehet lépni. Ezek elhelyezkedése véletlenszerű, de a tábla középpontjára szimmetrikus.
A játékosok felváltva lépnek. Egy lépés során a játékos kiválasztja egy saját baktériumát, majd egy üres célmezőt. Ha a célmező a baktérium szomszédja (átlósan is), a baktérium osztódik: az eredeti helyén marad, és a célmezőn egy új baktérium jelenik meg. Ha a célmező két mező távolságra van (azaz az 5 × 5-ös négyzet külső gyűrűjén), a baktérium átugrik oda, az eredeti mező pedig kiürül. A lépés után a célmezővel szomszédos összes ellenséges baktérium a lépő játékos színére változik. A baktérium kiválasztása után a program jelölje meg a lehetséges célmezőket.
Ha egy játékos nem tud lépni, passzolnia kell. A játék véget ér, ha a tábla megtelik, ha valamelyik játékosnak elfogynak a baktériumai, vagy ha egyik játékos sem tud lépni. Az a játékos győz, akinek több baktériuma van a táblán.
A program biztosítson lehetőséget új játék kezdésére a táblaméret megadásával (5 × 5, 7 × 7, 9 × 9), játék szüneteltetésére, valamint játék mentésére és betöltésére. Ismerje fel, ha vége a játéknak, és jelenítse meg, melyik játékos győzött (ha nem döntetlen). A program folyamatosan jelezze a játékosok baktériumainak számát, valamint külön-külön a két játékos gondolkodási idejét. A gondolkodási időt is mentsük el és töltsük be.

8. Pentago

Készítsünk programot, amellyel a *Pentago* játék alábbi, változtatható méretű változatát játszhatjuk. Adott egy n × n mezőből álló tábla, amely négy egyforma, (n/2) × (n/2) méretű negyedre oszlik. A játékosok felváltva lépnek. Egy lépés két részből áll: a játékos először lehelyez egy saját korongot egy üres mezőre, majd egy tetszőleges negyedet 90 fokkal elforgat az óramutató járásával megegyező vagy ellentétes irányba. A forgatás kötelező, és bármelyik negyedre vonatkozhat.
Az a játékos győz, akinek a lépés végén adott számú korongja áll egy vonalban (vízszintesen, függőlegesen vagy átlósan). Ha a forgatás után mindkét játékosnak kialakul ilyen vonala, vagy a tábla megtelik vonal nélkül, a játék döntetlen.
A program biztosítson lehetőséget új játék kezdésére a táblaméret megadásával (6 × 6 és 5 korong, 8 × 8 és 6 korong, 10 × 10 és 7 korong), játék szüneteltetésére, valamint játék mentésére és betöltésére. A program minden lépésnél jelezze, melyik játékos következik, és a lépés melyik része van hátra. Ismerje fel, ha vége a játéknak, jelenítse meg, melyik játékos győzött (ha nem döntetlen), és a táblán

jelölje meg a győztes vonalat. A program folyamatosan jelezze külön-külön a két játékos gondolkodási idejét, és ezt is mentsük el és töltsük be.

9. Torpedó

Készítsünk programot, amellyel a *Torpedó* (*Battleship*) játék alábbi, szonárral bővített változatát játszhatjuk. Mindkét játékos rendelkezik egy n × n mezőből álló táblával, amelyen hajóit helyezi el. A hajók 1 mező szélesek, 2–5 mező hosszúak, és vízszintesen vagy függőlegesen állhatnak. Két hajó nem érintkezhet, átlósan sem. A flotta összetétele a tábla méretétől függ. A játék elején mindkét játékos elhelyezi a hajóit úgy, hogy a másik ne lássa őket. Ehhez kézi elhelyezés és véletlenszerű automatikus elhelyezés is választható.
A játékosok felváltva lépnek. Lépésenként a játékos megjelöl egy mezőt az ellenfél tábláján, amelyre lő. A program jelzi, hogy a lövés talált, mellé ment, vagy elsüllyesztett egy hajót. Egy hajó akkor süllyed el, ha minden mezőjét eltalálták. Lövés helyett a játékos szonárt is használhat. A szonár egy kiválasztott 3 × 3-as négyzetről megmutatja, hány hajómező található benne, de azt nem, hogy melyek ezek. Használata után a játékos csak 3 saját kör elteltével használhatja újra.
Mivel a két játékos ugyanazt az eszközt használja, a játékosváltáskor a program takarja el a táblákat, és csak megerősítés után mutassa a soron következő játékos nézetét. Ez a nézet a saját táblából és az ellenfél tábláján eddig szerzett információkból áll.
A program biztosítson lehetőséget új játék kezdésére a táblaméret megadásával (8 × 8, 10 × 10, 12 × 12), valamint játék mentésére és betöltésére. Ismerje fel, ha vége a játéknak, azaz ha valamelyik játékos minden hajója elsüllyedt, és jelenítse meg, melyik játékos győzött. Játék közben folyamatosan jelenítse meg mindkét játékos még úszó hajóinak számát.

10. Ökoszisztéma

Készítsünk programot, amellyel a *Wa-Tor* ragadozó–zsákmány szimuláció alábbi, bővített változatát futtathatjuk. Adott egy n × n mezőből álló terület. Mezőin fű nőhet, és nyulak, illetve rókák élhetnek, mezőnként legfeljebb egy állat. A szimuláció adott időközönként lépésekben halad. Minden lépésben a lelegelt mezőkön adott valószínűséggel újranő a fű. A nyulak egy szomszédos mezőre lépnek, lehetőleg füvesre, és ha ott fű van, lelegelik. A rókák szomszédos nyúl esetén annak mezőjére lépnek és megeszik, egyébként véletlenszerűen mozognak.
Minden állatnak energiája van. Az energia lépésenként csökken, táplálkozáskor pedig nő. Ha elfogy, az állat elpusztul. Ha egy állat energiája egy küszöbérték fölé emelkedik, szaporodik: egy szomszédos üres mezőn új, azonos fajú állat jelenik meg, és a szülő energiája megfeleződik.

A szimuláció kezdeti paramétereit (a nyulak és a rókák kezdeti száma, a fű újranövési valószínűsége, a szaporodási küszöbök) a felhasználó adhatja meg. A program folyamatosan jelenítse meg a terület állapotát, valamint egy grafikonon a nyúl- és rókapopuláció alakulását az idő függvényében. Szüneteltetett állapotban a felhasználó egy mezőre kattintva elhelyezhet vagy eltávolíthat állatot, és a szimulációt lépésenként is léptetheti.
A program biztosítson lehetőséget új szimuláció indítására a terület méretének megadásával (10 × 10, 20 × 20, 30 × 30), a szimuláció szüneteltetésére, valamint sebességének állítására. Ismerje fel, ha valamelyik faj kihalt, ekkor a szimuláció álljon le, és jelenítse meg, hány lépésig maradt fenn mindkét faj. Ezen felül szüneteltetés alatt legyen lehetőség a szimuláció elmentésére, valamint betöltésére, a populációs előzményekkel együtt.